SWC3

Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering

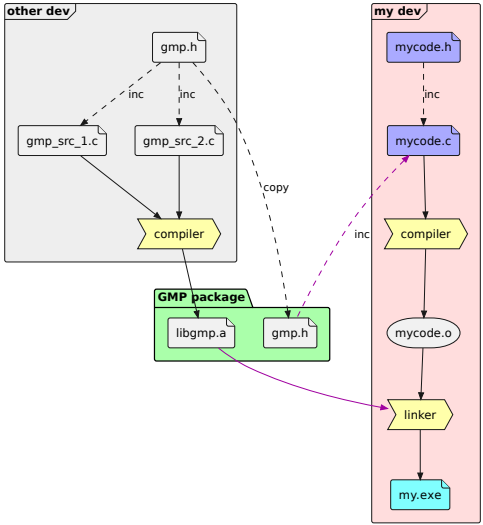


6. C Bibliotheken & Interoperabilität

Bibliotheken

- in der Praxis häufig Aufbau auf bestehende Funktionalität
- C Bibliotheken bestehen aus zwei Teilen
  - Header Dateien (`*.h`) mit Deklarationen
  - Implementierung
    - entweder als Quelltext (`*.c`)
    - als statische Bibliothek oder “Archiv” von Objektcode (`*.lib` oder `*.a`)
    - als dynamische Bibliothek, z.B. shared object (`*.so`) oder dynamic-link library (`*.dll`)
- Compiler benötigt Header-Dateien
- Linker benötigt Implementierung
- Beispiel: “GNU Multiple Precision Arithmetic Library” (GMP)[1]

[1] https://gmplib.org/



Einbinden von Bibliotheken: Beispiel GNU MP (i)

```
// file: mathdemo.c
#include <stdio.h>
#include <stdlib.h>
#include <gmp.h>
int main(int argc, char* argv[static argc+1]) {
  if (argc <= 2) exit(1);

  constexpr int BASE = 10;
  mpz_t x, y, result;
  mpz_init_set_str(x, argv[1], BASE);
  mpz_init_set_str(y, argv[2], BASE);
  mpz_init(result);

  mpz_mul(result, x, y);

  char *result_str =
    mpz_get_str(nullptr, BASE, result);
  printf("%s * %s = %s",
         argv[1], argv[2], result_str);
  // cleanup ...
}
```

- Rechnen mit beliebiger Genauigkeit
- Bibliothek *GNU multiple precision arithmetic library* (gmp)
  - Header `/usr/include/gmp/gmp.h`
  - Bibliothek `/usr/lib/gmp/libgmp.so`
- `mpz_t` ist ganze Zahl mit beliebiger Genauigkeit
- Einlesen der Kommandozeilenparameter `argv[1]` und `argv[2]`
  - z.B. `./mathdemo 32189045321 95639067`

```
32189045321 * 95639067 = 3078530262121155507
```

Einbinden von Bibliotheken: Beispiel GNU MP (ii)

- 1. Übersetzen der Quelltexdatei
  - `-I/usr/include/gmp` sucht Header in Verzeichnis `/usr/include/gmp`
  - Ergebnis ist Objekt-Datei `mathdemo.o`

```
gcc -I/usr/include/gmp -c mathdemo.c
ls -nh mathdemo.*
```

```
-rw-r--r-- 1 1000 1000  468 14. Sep 14:14 mathdemo.c
-rw-r--r-- 1 1000 1000 2.1K 14. Sep 14:48 mathdemo.o
```

Einbinden von Bibliotheken: Beispiel GNU MP (iii)

- 2a. Binden der Objektdatei schlägt fehl
  - Bibliothek und darin enthaltene Funktionen nicht gefunden

```
gcc -o mathdemo mathdemo.o 2>&1
```

```
/usr/bin/ld: mathdemo.o: in function `main':
mathdemo.c:(.text+0x50): undefined reference to `__gmpz_init_set_str'
/usr/bin/ld: mathdemo.c:(.text+0x6f): undefined reference to `__gmpz_init_set_str'
/usr/bin/ld: mathdemo.c:(.text+0x7b): undefined reference to `__gmpz_init'
/usr/bin/ld: mathdemo.c:(.text+0x92): undefined reference to `__gmpz_mul'
/usr/bin/ld: mathdemo.c:(.text+0xa8): undefined reference to `__gmpz_get_str'
collect2: error: ld returned 1 exit status
```

Einbinden von Bibliotheken: Beispiel GNU MP (iv)

- 2b. Angabe des Suchpfades mit `-L` und der Bibliothek mit `-l` erforderlich
  - Präfix `lib` und Suffix `.so` oder `.a` wird automatisch verwendet
  - Parameter `-lgmp` → Dateiname `libgmp.so`

```
gcc -L/usr/lib/gmp -o mathdemo mathdemo.o -lgmp
ls -nh mathdemo*
```

```
-rwxr-xr-x 1 1000 1000  16K 14. Sep 14:48 mathdemo
-rw-r--r-- 1 1000 1000  468 14. Sep 14:14 mathdemo.c
-rw-r--r-- 1 1000 1000 2.1K 14. Sep 14:48 mathdemo.o
```

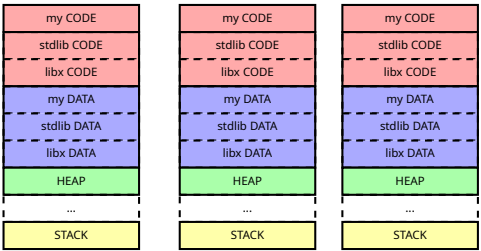
- Testlauf

```
./mathdemo 78903490364390378904  437830126749047038790432
```

```
78903490364390378904 * 437830126749047038790432 = 34546325187183251316402059593444702329846528
```

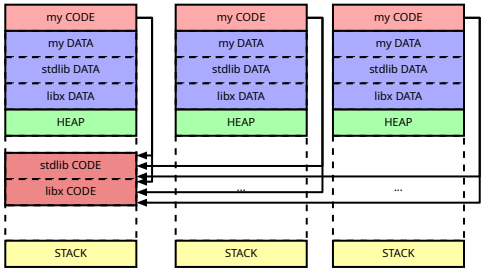
Statische und dynamische Bibliotheken (i)

- statische Bindung
  - gesamter Objektcode in einem Image
    - eigener Objektcode
    - eigene Bibliotheken
    - fremde Bibliotheken
  - Vorteile: einfach & schnell
  - Nachteile: großes Image, Redundanz im Speicher



Programme im Hauptspeicher

- dynamische Bindung
  - nur eigener Objektcode im Image
    - z.B. *dynamic-link library* (DLL)
    - oder *shared object* (SO)
  - Verweise auf Bibliotheken und Funktion
  - Vorteile: kleines Image, Wiederverwendung
  - Nachteil: extra Aufwand bei Aufruf



Programme im Hauptspeicher

```c
int main() { SHOW2(main, printf); }   // file: linking.c
```

Statische und dynamische Bibliotheken (ii)

Statische Bindung

- größeres Image
  - enthält allen Funktionen
- fixer Adresse der Funktionen
  - schneller

```
gcc -static -o linking-static linking.c
./linking-static
./linking-static
ls -sh linking-static
```

```
main = 0x402ec5;  printf = 0x406050
main = 0x402ec5;  printf = 0x406050
816K linking-static
```

Dynamische Bindung

- kleineres Image
  - Funktionen zur Laufzeit gesucht
- randomisierte Adressen
  - etwas langsamerer Aufruf

```
gcc -o linking-dynamic linking.c
./linking-dynamic
./linking-dynamic
ls -sh linking-dynamic
```

```
main = 0x55c5888f0149;  printf = 0x7f324ee5b8b0
main = 0x55b262933149;  printf = 0x7f303ba5b8b0
16K linking-dynamic
```

Interoperabilität (i)

- C wird oft als Brücke zwischen verschiedenen Programmiersprachen genutzt
  - einfache Sprache
  - sehr weit verbreitet
- Beispiel Aufruf von C aus Java mit Hilfe von `native`-Methode

```java
// file: CallC.java
class CallC {
  private static native int getTime();
}
```

- erzeugen der Schnittstelle für C mit Java-Compiler

```
javac -h . CallC.java # -> CallC.h
```

```c
// file CallC.h
JNIEXPORT jint JNICALL Java_CallC_getTime(JNIEnv*, jclass);
```

Interoperabilität (ii)

- Erstellen der Implementierung in C

```c
// file: callc.c
#include "CallC.h" // generated by 'java -h'

#include <stdio.h>
#include <time.h>
#include <jni.h>

// NOTE: ensure Java calling conventions
JNIEXPORT jint JNICALL Java_CallC_getTime(
    [[maybe_unused]] JNIEnv* env,
    [[maybe_unused]] jclass c) {
  printf("Hello C!\n");
  return time(nullptr);
}
```

- Erstellen einer shared library aus C-Code (GCC Option `-shared`)

```
gcc -std=c23 -pedantic -Wall -Wextra \
    -I $JAVA_HOME/include \
    -I $JAVA_HOME/include/linux \
    -shared \
    -o libcallc.so \
    callc.c
```

- Ergebnis ist Datei `libcallc.so`

Interoperabilität (iii)

- Laden der shared Library in Java
- Aufruf wie normale Methode

```java
class CallC {

  private static native int getTime();

  static {
    System.loadLibrary("callc");
  }

  public static void main(String[] args) {
    System.out.println("Hello Java");
    int time = getTime();
    System.out.println("C time is " + time);
  }
}
```

- Angabe des Suchpfads für Bibliotheken (`java.library.path`) bei Aufruf[1]

```
javac CallC.java
java --enable-native-access=ALL-UNNAMED \
    -Djava.library.path=. \
    CallC
```

```
Hello Java
C time is 1789390126
Hello C!
```

[1] aktuelles Verzeichnis ist »`.`«